ANALYSIS BLUEPRINT · CAPACITY PLANNING

Fleet or Flow?

A decision-first plan for the bike-sharing case: profile the data, test what suppresses demand, forecast it probabilistically, simulate how many bikes are actually on the road, and give operations a question-answering desk over a pre-computed demand cube.

Instructions for Claude Code

This blueprint is the full specification. Place it in an empty folder with `hour.csv`, `day.csv`, `Readme.txt` and the case-study PDF, then paste the launch prompt in Part 9. Build one **workstream** at a time, run its quality gate, report results, and pause for approval before the next one.

PART 1

The decision this work must support

The CEO's question — *buy more bikes?* — is really a question about which constraint binds. Two observations (riders find no bike; bikes sit idle) can coexist under three different explanations, and each one implies a different action. The whole analysis is organised to tell them apart.

Explanation	What it would look like in data	Action it implies
A. Fleet too small	Demand high across many hours; peak-to-average ratio modest; simulated bikes in use close to fleet for long periods	Buy bikes
B. Wrong time (temporal mismatch)	Sharp, short commute spikes; most of the day far below peak; large share of fleet idle outside 2–4 hours	Size the fleet to a service level, not the max; shift idle hours to maintenance
C. Wrong place (spatial mismatch)	Cannot be seen in aggregate data — needs station data; indirect hints from one-directional commute flows	Rebalancing, docking incentives
D. Demand is fragile	Large drops with weather, especially among occasional users	Don't size for fair-weather leisure peaks; plan fleet on the stable commuter base

The data can confirm or rule out A, B and D. It cannot directly test C, so the plan ends with a field experiment that can. The deliverable is a **decision memo**, not just a model.

PART 2

What we are given — and what we are not

Item	Detail
Source	Capital Bikeshare, Washington D.C., 2011-01-01 to 2012-12-31, with weather (Readme.txt)
hour.csv	17,379 rows × 17 columns — one row per observed hour
day.csv	731 rows × 16 columns — daily totals (equals hourly sums; verify)
Targets	casual (non-members), registered (members), cnt = casual + registered
Scaled weather	temp ×41 → °C, atemp ×50 → °C, hum ×100 → %, windspeed ×67 → km/h (per Readme)
Missing by design	station IDs, trip origin/destination, trip duration, fleet size, dock status, prices

Profile facts to reproduce exactly (already checked on these files)

- Calendar coverage: 731 days × 24 = 17,544 possible hours; 165 are absent from hour.csv.
- No nulls, no duplicate rows; casual + registered = cnt everywhere; hourly totals match day.csv for all 731 days.
- Sentinel zeros: windspeed = 0 in 2,180 rows; hum = 0 in 22 rows.
- weathersit = 4 occurs in 3 rows only.
- season 1 spans January to mid-March (not 'spring' as the Readme label suggests).
- temp and atemp correlate at ≈ 0.99.
- Rentals grew from 1,243,103 (2011) to 2,049,576 (2012); peak hour 977 rentals on 2012-09-12 at 18:00; lowest day 22 rentals on 2012-10-29 (Hurricane Sandy).

Censoring caveat

Recorded rentals are a *lower bound* on demand: a rider who found no bike leaves no trace. Every forecast in this project is a forecast of **served** demand. Workstream 2 includes a probe for supply-capped hours, and the memo must state the direction of the bias (true peak demand is likely higher, not lower).

PART 3

Engineering approach and stack

Pipeline-first, not notebook-first: each workstream is a script that reads versioned inputs and writes artifacts. A single `Makefile` rebuilds everything from raw data. Notebooks exist only as thin storytelling layers that load finished artifacts.

Concern	Choice	Reason
Environment	uv + pyproject.toml, Python 3.11	Fast, locked, reproducible installs
Orchestration	Makefile targets per workstream	One command rebuilds the project end to end
Data engine	DuckDB + Polars (pandas only where a library needs it)	SQL-native analysis; the same database later serves the app
Data validation	pandera schemas	Data contract enforced in code, failures are loud
Time-series analysis	statsmodels (STL, GLMs), scipy	Decomposition, count models, tests
Pattern discovery	scikit-learn (KMeans, silhouette)	Day-type archetypes
Forecasting	CatBoost (Tweedie / Poisson loss), statsmodels NegativeBinomial	Native categoricals; count-aware loss; no extra macOS system libraries
Uncertainty	MAPIE conformal prediction	Distribution-free intervals with guaranteed coverage
Simulation	NumPy vectorised Monte Carlo	Bikes-in-use and fleet service levels
Explainability	CatBoost SHAP values, partial dependence	Driver analysis
App	Gradio (tabs) on top of a Python package	Chat + explorer + simulator in one UI
LLM access	LiteLLM → Gemini / Claude / local Ollama model	One interface, provider switch by config; offline-capable
SQL safety	sqlglot	Parse and whitelist LLM-generated SQL before execution
Docs search	DuckDB full-text search (BM25)	Lightweight retrieval over project notes
Charts	Altair (static export via vl-convert)	Declarative, consistent figures
Quality	pytest, ruff	Tests and linting

Claude Code must check the current docs for LiteLLM, MAPIE, CatBoost and Gradio APIs before use, and keep all provider/model names in `config.yaml` and keys in `.env`.

PART 4

Project layout

```
fleet-or-flow/
├── Makefile                # one target per workstream + all
├── pyproject.toml          # dependencies (managed with uv)
├── config.yaml             # dates, seeds, cost assumptions, trip-duration prior, LLM settings
├── .env.example
├── raw/                    # hour.csv, day.csv, Readme.txt  (read-only)
├── warehouse/
│   └── bikes.duckdb        # cleaned tables, features, forecasts, demand cube, notes index
├── bikeops/                # importable package
│   ├── contract.py         # pandera schemas
│   ├── audit.py            # WS1 checks + repairs
│   ├── signals.py          # feature construction (single source of truth)
│   ├── archetypes.py       # daily-profile clustering
│   ├── inference.py        # WS2 models and tests
│   ├── forecast.py         # WS3 training, backtests, conformal intervals
│   ├── concurrency.py      # Monte Carlo bikes-in-use
│   ├── fleet.py            # service-level & cost-based fleet sizing
│   ├── cube.py             # materialise the demand cube
│   └── desk/               # WS4 assistant
│       ├── router.py       # intent routing
│       ├── sql_agent.py    # text-to-SQL with guardrails
│       ├── tools.py        # simulate_fleet, explain_driver, search_notes
│       └── ui.py           # Gradio app
├── artifacts/
│   ├── figures/  tables/  models/
├── docs/
│   ├── data_audit.md  hypothesis_brief.md  model_card.md  decision_memo.md  decision_log.md
├── storybook/              # 3 thin notebooks reading artifacts (optional viewing layer)
└── tests/
```

WORKSTREAM 1

Data contract, audit and signal design (Task 1)

1.1 Data contract (bikeops/contract.py)

Encode the Readme as a pandera schema: allowed ranges and categories for every column, the identity casual + registered = cnt, one row per (date, hour), and dteday/yr/mnth/weekday agreement with the actual calendar. Running the contract on raw data is expected to *fail* on the sentinel zeros — that failure is evidence for the audit, captured into `artifacts/tables/contract_violations.csv`.

1.2 Audit (bikeops/audit.py → docs/data_audit.md)

Check	Method	Repair
Calendar gaps	Anti-join against a full 17,544-hour spine in DuckDB; classify gaps as night lull (00–05, isolated) or outage (daytime / consecutive)	Night lulls → 0 rentals; outage hours → left missing and excluded from training; flag both
Sentinel zeros	windspeed = 0, hum = 0	Set null; fill by same-day linear interpolation, else month-hour median; keep an `imputed` flag
Sparse category	weathersit frequency table	Fold 4 into 3

Check	Method	Repair
Label semantics	season × month crosstab; workingday vs weekday/holiday logic	Use meteorological season derived from month in modelling; document Readme mismatch
Redundancy	temp vs atemp correlation, VIF	Keep temp only
Anomalous days	Robust z-score of daily residual after STL (|z| > 3.5)	Tag as `disruption_day`; keep for analysis, test sensitivity in modelling
Cross-file	Hourly sum vs day.csv	Assert equal

1.3 Understanding the demand signal

a) **STL decomposition** of daily totals (weekly period): separate trend, weekly cycle and remainder. Quantify the trend (growth per month) and the size of the weekly cycle.

b) **Day archetypes**: represent each day by its 24-hour *shape* (hourly share of the day's total), cluster with KMeans (k = 2–5, pick by silhouette). Expected: a two-peak commute shape and a single midday-hump leisure shape, possibly a flattened bad-weather shape. Cross-tabulate archetype vs workingday, weathersit, month; list the 'misfits' (e.g. working days that look like weekends) and explain them.

c) **Segment split**: share of casual vs registered by archetype and by hour.

d) **Weather response curves**: binned mean demand vs temperature (°C) per segment, showing the rise and the hot-weather plateau; demand by weathersit per segment.

e) **Concentration**: what share of weekly rentals falls in the busiest 10% of hours (a Lorenz-style curve). This is the core visual for explanation B.

1.4 Signals (bikeops/signals.py)

Signal	Construction	Why
hour, dow, month, met_season	From the timestamp	Calendar structure; CatBoost treats them as categorical
calendar_class	workday / weekend / holiday / bridge day (workday between holiday and weekend)	Bridge days behave like weekends
archetype_prior	Most likely archetype for (dow, month, holiday) from training days only	Encodes the day's expected shape without leaking the day's own data
temp_c, hum_pct, wind_kmh	Rescaled, imputed	Interpretable units
comfort_index	temp_c penalised for humidity above 70% and wind above 25 km/h (document formula)	One 'is it pleasant to ride' signal
wet	weathersit == 3	Rain/snow switch
daylight	1 if hour between approx. sunrise and sunset for D.C. that date (astral library or fixed monthly table)	Evening demand depends on light
growth_index	Months since start	Trend; frozen at last training value for later dates

Never used as inputs: instant, cnt, casual, registered, atemp, any same-hour target transform.

Quality gate — WS1

- Profile facts in Part 2 reproduced; contract violations file produced; repaired data passes the contract.
- docs/data_audit.md has a check → finding → action → impact table.
- Figures: STL panel, archetype centroids, segment hour profiles, temperature response, concentration curve.

- tests: signals deterministic, no target leakage, archetype_prior built from training days only.

WORKSTREAM 2

Hypothesis: who stays home when the weather turns (Task 2)

Hypothesis

H1. Occasional riders (casual) are far more weather-sensitive than members (registered): holding hour, calendar class, month and trend fixed, wet weather reduces casual rentals by a substantially larger proportion than registered rentals (ratio of rate-ratios < 0.75).
H0. The wet-weather rate-ratio is the same for both segments.
Why it matters for the CEO. If the peak that drives fleet size is the fair-weather leisure crowd, buying bikes to serve it leaves them idle on every cool or rainy day — part of the 'unused bikes' picture. The fleet should be planned on the stable member base plus a service-level allowance.

2.1 Test design (bikeops/inference.py)

a) **Count model.** Long format (hour × segment). Negative Binomial GLM: `rentals ~ segment × wet + segment × temp_c + segment × C(hour) × C(calendar_class) + C(month) + growth_index`. Cluster-robust standard errors by date (hours within a day are not independent). The test statistic is the `segment × wet` interaction, reported as a ratio of rate-ratios with 95% CI.

b) **Matched comparison** (quasi-experimental check that does not depend on the model form). Pair each wet hour with dry hours of the same hour-of-day, calendar class and month (nearest date, same year). Compute the per-pair log-ratio of wet vs dry for each segment; compare segments with a Wilcoxon signed-rank test and a bootstrap CI over days.

c) **Temperature elasticity** as a secondary result: percent change in each segment per +1 °C in the 5–25 °C band.

d) **Pre-registered decision rule**, written into docs/hypothesis_brief.md before results are produced: reject H0 if both methods give a ratio of rate-ratios below 0.75 with CIs excluding 1.

e) **Robustness**: each year separately, excluding disruption days, using weathersit 2 (mist) as a second 'treatment'.

2.2 Supply-cap probe (exploratory, clearly labelled)

If the system was bike-constrained, the best-condition peak hours should pile up just below a ceiling that grows with the fleet over time. For working-day 08:00 and 17–18:00 hours in comfortable dry weather, inspect the upper tail by month: rolling 95th percentile, and whether residual variance shrinks as predicted demand rises (a sign of truncation). Report it as suggestive evidence only.

2.3 Field experiment to settle what the data cannot

Capacity-elasticity test. In a switchback design over 8 weeks, randomly assign matched station clusters each week to (i) normal fleet, (ii) 10% of bikes withheld at the depot, or (iii) 10% extra bikes from the depot placed before the AM peak. Outcomes: rentals per cluster-hour, zero-bike minutes, rentals per bike. If removing 10% of bikes does not reduce rentals, the fleet has slack and buying more is wasteful; if adding bikes raises rentals only in specific clusters, the problem is placement (explanation C). Power analysis with the hourly variance estimated here; state the minimum detectable effect.

Quality gate — WS2

- docs/hypothesis_brief.md: hypothesis, rule, both test results with CIs, robustness table, limitations (observational, censoring, no location), the probe, and the experiment with power

calculation.
- Every number generated by code and stored in artifacts/tables/.

WORKSTREAM 3

Probabilistic forecast and fleet sizing (Task 3)

3.1 Forecast design (bikeops/forecast.py)

- **Targets**: casual and registered hourly counts, modelled separately with count-aware loss (Tweedie, variance power tuned around 1.2–1.6; compare with Poisson). Total = sum.
- **Backtesting**: walk-forward evaluation. Expanding training window, forecast the next 4 weeks, step 4 weeks, starting 2012-03-01 through 2012-12-31 (10 folds). Hyper-parameters tuned on the first folds only; later folds are the honest score.
- **Two-level consistency check**: train a daily model on day.csv; compare the sum of hourly forecasts with the daily forecast per fold. Large disagreement flags a problem. Optionally reconcile (scale hourly shapes to the daily total) and keep it if it improves the score.

Contender	Role
Seasonal naive	Same hour, same calendar class, previous comparable week
Profile × level	Archetype hourly shape × trailing 4-week daily mean
Negative Binomial GLM	Interpretable statistical benchmark (from WS2 specification)
CatBoost Tweedie (champion candidate)	Signals from WS1, categorical calendar features
CatBoost + recent history (diagnostic)	Adds lagged demand; shows the value of live data feeds, not used by the desk

3.2 Scoring

- WAPE (weighted absolute % error) — the headline business metric.
- Mean Poisson deviance and RMSE on counts.
- Pinball loss at 0.1 / 0.5 / 0.9 and empirical coverage of the 80% and 90% intervals.
- Peak-window error (weekday 07–09, 16–19) reported separately; fleet decisions depend on it.
- Skill score vs seasonal naive; champion must beat every simpler contender on peak-window WAPE.

3.3 Uncertainty

Wrap the champion with **MAPIE conformal prediction** (time-series aware, residuals from the walk-forward folds) to produce 80% and 90% intervals. Check coverage separately for peak and off-peak hours; if peaks are under-covered, calibrate intervals per hour group (Mondrian conformal).

3.4 From rentals to bikes on the road (bikeops/concurrency.py)

Rentals per hour are not bikes needed — what matters is how many bikes are out *at the same time*. With no trip durations in the data, use an explicit prior and simulate:

```
for each simulated day (N = 2,000 draws per day type x month):
    hourly rentals  ~ forecast predictive distribution (conformal quantiles / NB draws)
    start minute    ~ uniform within the hour
    trip duration   ~ LogNormal(median = 18 min, p90 = 45 min)   # prior from config.yaml
    bikes_in_use(t) = number of trips active at minute t
record: daily peak concurrency, minutes above each fleet level, average utilisation
```

Sensitivity: median trip 12 / 18 / 25 minutes. The trip prior is the largest assumption in the project and must be shown as such.

3.5 Fleet sizing (bikeops/fleet.py)

a) **Service-level view**: fleet F(α) = the α-quantile of daily peak concurrency (α = 0.90, 0.95, 0.99), multiplied by a *spatial friction* factor (1.2–1.5) for bikes stranded at the wrong docks.

b) **Cost view (newsvendor)**: with cost of a missed ride C_u and daily holding cost of a bike C_o (both placeholders in config.yaml, shown with sensitivity), the optimal service level is $C_u / (C_u + C_o)$; read the fleet off the concurrency distribution at that level.

c) **Idle-capacity view**: for the recommended fleet, hourly utilisation curve and share of bike-hours idle — quantifying the CEO's 'unused bikes'.

d) **Fleet vs flow comparison**: bikes required at a 95% service level with friction 1.5 (poor rebalancing) vs 1.2 (good rebalancing). The difference is the number of bikes that better rebalancing substitutes for — the key number for the memo.

Quality gate — WS3

- artifacts/tables/backtest_scores.csv with all contenders × folds; champion justified in docs/model_card.md (intended use, data range, metrics, limitations, assumptions).
- Interval coverage within ±3 points of nominal on the backtest.
- Fleet tables: service-level grid, cost-based optimum, friction comparison; figures for concurrency distribution and utilisation curve.
- tests: concurrency simulation reproducible with seed; fleet monotone in α and in trip median.

WORKSTREAM 4

Operations Q&A desk (Task 4)

4.1 Idea: query a demand cube, don't call a model live

All model output is materialised once into a **demand cube** table in DuckDB. The assistant answers questions by querying it with SQL. Answers are fast, auditable (the SQL is shown) and identical every time.

Table / view	Grain	Columns (main)
demand_cube	month × calendar_class × hour × weather_band (dry-mild, dry-hot, dry-cold, wet) × trip_median	p10, p50, p90 of casual / registered / total rentals; p50 and p95 bikes in use; bikes at 95% service
history_hourly	Cleaned observed hours	date, hour, calendar_class, weather, temp_c, casual, registered, cnt, archetype
history_daily	Day	date, totals, archetype, disruption flag, STL trend
fleet_scenarios	service level × friction × trip median	Required fleet, idle share, cost per day
notes	Paragraph	Text chunks from docs/*.md and Readme.txt with source and heading

For a specific calendar date, a small resolver maps it to (month, calendar_class) using a US federal holiday calendar, and weather words ('rain', 'hot', 'nice day') to a weather band. Dates beyond 2012 are answered at the 2012 demand level and the answer says so.

4.2 Assistant architecture (bikeops/desk/)

```
question
  └─ router (LLM, structured output) ──► intent ∈ {lookup, compare, what_if_fleet, explain, history}
       lookup / compare / history ──► sql_agent: LLM writes SQL over whitelisted views
                                        └─ sqlglot: parse → SELECT-only → allowed tables/columns → LIMIT 500
                                        └─ DuckDB read-only connection → result rows
       what_if_fleet ──► simulate_fleet(service_level, friction, trip_median)  (reads fleet_scenarios / reruns)
       explain       ──► search_notes(query)  (DuckDB BM25 over notes) + SHAP summary table
  └─ answer writer (LLM) ──► uses ONLY returned rows/notes, cites table or document, states assumptions
```

- **Providers via LiteLLM**: Gemini or Claude with an API key; a local Ollama model as the offline option for the demo. Provider/model set in config.yaml.
- **Self-check**: if SQL fails, the error is returned to the LLM once for a corrected query; after that, the desk says it could not answer.
- **Semantic layer**: the SQL prompt receives table descriptions, column meanings, allowed values and 6 worked question→SQL examples. No raw schema guessing.
- **Numbers come only from query results**; the writer is instructed to quote them, never compute new ones.

4.3 Interface (Gradio, three tabs)

Tab	Contents
Ask the desk	Chat; under each answer: the SQL or tool call, a result table, and an Altair chart
Demand explorer	Pick month, day type, weather → hourly bands (p10–p90) for each segment
Fleet simulator	Sliders: service level, friction, trip median, costs → required fleet, idle share, chart of bikes in use

4.4 Questions the desk must answer well (acceptance set)

a) How many bikes should be available at 6 PM on a rainy weekday in November?
b) Which hours on a hot Saturday in July need the most bikes?
c) How much smaller is casual demand on wet days compared with dry ones?
d) What fleet do we need for a 95% service level if trips average 25 minutes?
e) How many bikes would better rebalancing save us?
f) What happened on 29 October 2012?
g) Why do you treat windspeed zero as missing?
h) What share of the fleet sits idle at 3 AM on a weekday?

Quality gate — WS4

- All 8 acceptance questions answered correctly with an API provider and with the local model; results match direct SQL. Store them as tests/test_desk_acceptance.py with expected values from the cube.
- Guardrail tests: DROP/UPDATE/ATTACH statements rejected; unknown tables rejected; long results truncated.
- Router accuracy ≥ 90% on a labelled set of 30 questions (stored in tests/).

PART 5

Decision memo (docs/decision_memo.md)

Two pages, written for the CEO, structured as **Answer → Evidence → Options → Risks → Next step**:

- **Answer** in one paragraph: which of explanations A–D the evidence supports.
- **Evidence**: concentration curve, archetypes, segment weather sensitivity, fleet-vs-flow number.
- **Options table**: (1) buy bikes, (2) improve rebalancing, (3) service-level-based fleet with maintenance in idle windows, (4) do nothing — each with estimated bikes, cost placeholder and risk.
- **Risks and unknowns**: censoring, missing location data, trip-duration prior, data age (2011–2012).
- **Next step**: the capacity-elasticity experiment and the data feeds to start collecting.

- The memo must follow the numbers. If the evidence points to explanation A, the memo recommends buying bikes.

Also write `docs/decision_log.md` throughout: every analytical choice with date, options considered, choice and reason. It doubles as interview preparation.

PART 6

Runbook

```
uv sync                                  # install dependencies
cp .env.example .env                     # add GEMINI_API_KEY or ANTHROPIC_API_KEY (optional)
ollama pull <local-model>                # optional offline model, name set in config.yaml
make all                                 # full pipeline, raw data -> memo
make app                                 # launches the Gradio desk
make test                                # pytest + ruff
```

PART 7

Rules of engagement for Claude Code

- One workstream at a time; finish its quality gate, summarise results with the key figures, then stop.
- All logic in the bikeops package; storybook notebooks only display artifacts.
- Time-respecting validation only; no information from the future in any feature or interval.
- Seeds fixed in config.yaml; `make all` from a clean clone must reproduce every artifact.
- No hard-coded results in docs — generate tables and insert them from artifacts.
- If any profile fact in Part 2 does not reproduce, stop and report before continuing.
- Commit after each workstream with a descriptive message.

PART 8

Assumptions register (start state)

Assumption	Default	Tested range	Where it bites
Trip duration median / p90	18 / 45 min	12–25 min median	Bikes in use, fleet size
Spatial friction	1.3	1.2–1.5	Fleet vs flow number
Service level	95%	90–99%	Fleet size
Missed-ride cost C_u, bike daily cost C_o	Placeholders	Grid	Cost-optimal service level
Night gap hours = zero rentals	Yes	Excluded instead	Night profile, backtest
Demand level for post-2012 dates	Frozen at 2012	—	Desk answers for future dates
Readme scaling for temperature	×41	Min-max alternative	Temperature response, comfort index

PART 9

Launch prompt for Claude Code

```
You are building the project described in BLUEPRINT.pdf (in this folder).
Read the whole document before creating any file.

Start with Part 4 (layout) and Part 3 (stack): scaffold the repository with uv,
move the CSVs and Readme into raw/, create config.yaml and the Makefile.

Then build WORKSTREAM 1 only. Run it, reproduce the profile facts from Part 2,
show me the audit table, the archetype result and the quality-gate checklist,
and wait for my go-ahead. Continue with Workstreams 2, 3, 4 and Part 5 the same
way, one at a time.

Follow Part 7 strictly. Keep docs/decision_log.md updated as you go.
```